

Análisis de Algoritmos 2025/2026

Práctica 3

Shaofan Xu y Alejandro Zheng (120/127)

Código	Gráficas	Memoria	Total

1. Introducción.

El objetivo de esta práctica es analizar experimentalmente el comportamiento y la eficiencia de los algoritmos de recuperación de información sobre un Diccionario TAD, concretamente la Búsqueda Lineal, la Búsqueda Binaria y la Búsqueda Lineal Auto-organizada.

Para ello, se implementarán dichos algoritmos en lenguaje C utilizando tablas como estructura de datos subyacentes y se evaluará su rendimiento sobre diccionarios de distintos tamaños, midiendo tanto el tiempo de ejecución como el número de operaciones básicas realizadas. Se generarán gráficos que permitirán comparar los resultados empíricos con el comportamiento teórico esperado, identificando las diferencias de costo entre mantener una tabla ordenada (para búsqueda binaria) o desordenada.

También se discutirán los resultados en términos de eficiencia temporal y se analizará la influencia de la distribución de las claves (uniforme frente a sesgada) en el rendimiento de la Búsqueda Auto-organizada, evaluando cómo la reordenación dinámica de elementos optimiza el acceso a los datos más frecuentes.

2. Objetivos

2.1 Apartado 1

El objetivo de apartado 1 es implementar todas las funciones de módulo de search. Además, implementar los algoritmos de búsqueda lineal, búsqueda binaria y la búsqueda lineal Auto-organizada. También así implementamos el TAD diccionario y sus correspondientes funciones como **init**, **insert**, **free**, etc.

Después comprobar el funcionamiento de estas funciones usando el fichero llamado `exercise1.c`. De esta forma, comprobamos el buen funcionamiento de los algoritmos.

2.2 Apartado 2

El objetivo de apartado 2 es implementar las funciones de medición de tiempo para los algoritmos de búsqueda. Así funciones como: **short average_search_time**, **short generate_search_times**. Después utilizar el fichero `exercise2.c` para medir tiempo de ejecución de los tres algoritmos implementados previamente.

3. Herramientas y metodología

Para el desarrollo de esta práctica se ha utilizado un entorno de desarrollo basado en Linux (Ubuntu 24.04.3), aprovechando las herramientas de la línea de comandos que ofrece este sistema.

Las herramientas principales empleadas han sido:

- **Editor de Código:** Visual Studio Code.
- **Compilador:** gcc, utilizando los flags: -Wall para activar todas las advertencias, -pedantic para generar la información necesaria para la depuración, -ansi para usar el standard C89. Y, además -O3 para optimizar el programa.
- **Análisis de Memoria:** Valgrind, una herramienta fundamental para detectar fugas de memoria y accesos a memoria inválidos.
- **Automatización:** nace, mediante un fichero Makefile, para automatizar el proceso de compilación y ejecución de los distintos ejercicios.
- **Visualización de Datos:** Gnuplot, para generar las gráficas a partir de los ficheros de resultados y poder realizar un análisis visual comparativo del rendimiento de los algoritmos.

Los dos apartados hemos utilizados las mismas herramientas para desarrollar.

4. Código fuente

4.1 Apartado 1

PDICT init_dictionary (int size,char order);

```

1. PDICT init_dictionary (int size, char order)
2. {
3.     PDICT re;
4.
5.     assert(size > 0);
6.
7.     re = malloc(1*sizeof(DICT));
8.     if (re == NULL)
9.     {
10.         return NULL;
11.     }
12.
13.     re->size = size;
14.     re->order = order;
15.     re->n_data = 0;
16.
17.     re->table = malloc(size*sizeof(re->table[0]));
18.     if (re->table == NULL)
19.     {
20.         free(re);
21.         return NULL;
22.     }
23.
24.     return re;
25.
26. }
```

void free_dictionary(PDICT pdict);

```
1. void free_dictionary(PDICT pdict)
2. {
3.     assert(pdict!=NULL);
4.     assert(pdict->table!=NULL);
5.     free(pdict->table);
6.     free(pdict);
7. }
```

int insert_dictionary(PDICT dict,int key);

```
1. int insert_dictionary(PDICT pdict, int key)
2. {
3.     int ob;
4.     int j;
5.
6.     assert(pdict != NULL);
7.
8.     if (pdict->n_data == pdict->size)
9.     {
10.         return ERR;
11.     }
12.
13.     pdict->table[pdict->n_data] = key;
14.     pdict->n_data++;
15.
16.     ob = 0;
17.
18.     if (pdict->order == SORTED)
19.     {
20.         j = pdict->n_data - 2;
21.
22.         while(j >= 0 && pdict->table[j] > key)
23.         {
24.             pdict->table[j+1] = pdict->table[j];
25.             j--;
26.             ob++;
27.         }
28.
29.         pdict->table[j+1] = key;
30.
31.     }
32.
33.     return ob;
34. }
```

int massive_insertion_dictionary (PDICT pdict,int *keys,int n_keys);

```
1. int massive_insertion_dictionary (PDICT pdict,int *keys, int n_keys)
2. {
3.     int obs,i;
4.     assert(pdict!=NULL);
5.     assert(pdict->table!=NULL);
6.     assert(keys!=NULL);
7.     assert(n_keys>=0);
8.
9.     obs=0;
10.    for(i=0;i<n_keys;i++)
11.    {
12.        obs+=insert_dictionary(pdikt,keys[i]);
13.    }
```

```
14.  
15.     return obs;  
16. }
```

int search_dictionary(PDICT pdict,int key,int *ppos,pfunc_search method);

```
1. int search_dictionary(PDICT pdict, int key, int *ppos, pfunc_search method)  
2. {  
3.     int ob;  
4.  
5.     assert(pdict != NULL);  
6.     assert(method != NULL);  
7.     if (pdict->n_data == 0)  
8.     {  
9.         *ppos = NOT_FOUND;  
10.        return 0;  
11.    }  
12.    ob = method(pdict->table,0,pdict->n_data-1, key, ppos);  
13.  
14.    return ob;  
15. }
```

int bin_search(int *table,int F,int L,int key,int *ppos);

```
1. int bin_search(int *table,int F,int L,int key, int *ppos)  
2. {  
3.     int mid,left,right,obs;  
4.  
5.     assert(table!=NULL);  
6.     assert(F>=0);  
7.     assert(L>=F);  
8.     assert(ppos!=NULL);  
9.  
10.    left=F;  
11.    right=L;  
12.    obs=0;  
13.  
14.    while(left<=right)  
15.    {  
16.        mid=(left+right)/2;  
17.        obs++;  
18.        if(table[mid]==key)  
19.        {  
20.            *ppos=mid;  
21.            return obs;  
22.        }else if(table[mid]<key)  
23.        {  
24.            left=mid+1;  
25.        }else{  
26.            right=mid-1;  
27.        }  
28.    }  
29.    *ppos=NOT_FOUND;  
30.    return obs;  
31. }
```

int lin_search(int *table,int F,int L,int key,int *ppos);

```
1. int lin_search(int *table,int F,int L,int key, int *ppos)
2. {
3.     int ob;
4.     int i;
5.     assert(table != NULL);
6.     assert(F<=L);
7.     assert(ppos != NULL);
8.
9.     ob = 0;
10.
11.     for (i = F; i <=L; i++)
12.     {
13.         ob++;
14.         if (table[i] == key)
15.         {
16.             *ppos = i;
17.             return ob;
18.         }
19.     }
20.
21.
22.     *ppos = NOT_FOUND;
23.
24.     return ob;
25. }
```

int lin_auto_search(int *table,int F,int L, int key,int *ppos);

```
1. int lin_auto_search(int *table,int F,int L,int key, int *ppos)
2. {
3.     int i,obs;
4.     assert(table!=NULL);
5.     assert(F>=0);
6.     assert(L>=F);
7.     assert(ppos!=NULL);
8.
9.     obs=0;
10.
11.     for(i=F;i<=L;i++)
12.     {
13.         obs++;
14.         if(table[i]==key)
15.         {
16.             if(i>F)
17.             {
18.                 swap(&table[i],&table[i-1]);
19.                 *ppos=i-1;
20.             }else{
21.                 *ppos=i;
22.             }
23.             return obs;
24.         }
25.     }
26.     *ppos=NOT_FOUND;
27.     return obs;
28. }
```

4.2 Apartado 2

short average_search_time(pfunc_search metodo, pfunc_key_generator generator, char order, int N, int n_times, PTIME_AA ptime);

```
1.  short average_search_time(pfunc_search metodo, pfunc_key_generator generator, char
order, int N, int n_times, PTIME_AA ptime)
2.  {
3.      int i;
4.      PDICT pdict;
5.      int *permutation;
6.      int *keys;
7.      clock_t start, end;
8.      double mean_time;
9.      long mean_ob=0;
10.     int max_ob, min_ob, ob;
11.     int ppos;
12.
13.     /*Error comprobation*/
14.     assert(metodo!=NULL);
15.     assert(N>0);
16.     assert(n_times>0);
17.     assert(ptime!=NULL);
18.
19.     /* Create dictionary and verify its not NULL*/
20.     pdict=init_dictionary(N,order);
21.     if(pdict==NULL) return ERR;
22.
23.     /* Create a permutations */
24.     permutation=generate_perm(N);
25.     if(permutation==NULL)
26.     {
27.         free_dictionary(pdict);
28.         return ERR;
29.     }
30.     if(massive_insertion_dictionary(pdict,permutation,N)==ERR)
31.     {
32.         free_dictionary(pdict);
33.         free(permutation);
34.         return ERR;
35.     }
36.
37.     /*Create a table of size n_times*N*/
38.     keys=malloc(sizeof(keys[0])*n_times*N);
39.     if(keys==NULL)
40.     {
41.         free_dictionary(pdict);
42.         free(permutation);
43.         return ERR;
44.     }
45.     generator(keys,n_times*N,N);
46.
47.     /* Count the time used to search the keys*/
48.     start=clock();
49.
50.     for(i=0;i<n_times*N;i++)
51.     {
52.         ob=search_dictionary(pdict,keys[i],&ppos,metodo);
53.         if(ob==ERR)
54.         {
55.             free_dictionary(pdict);
56.             free(permutation);
57.             free(keys);
58.             return ERR;
59.         }
```

```

60.     if(i==0)
61.     {
62.         max_ob=min_ob=ob;
63.     }
64.     mean_ob+=ob;
65.     max_ob=max_ob<ob?ob:max_ob;
66.     min_ob=ob<min_ob?ob:min_ob;
67. }
68. end=clock();
69.
70. mean_time=((double)end-start)/(n_times*N);
71.
72.
73. /* Asignation of values to ptime */
74. ptime->n_elems=n_times*N;
75. ptime->N=N;
76. ptime->time=mean_time/CLOCKS_PER_SEC;
77. ptime->average_ob=((double)mean_ob)/(n_times*N);
78. ptime->min_ob=min_ob;
79. ptime->max_ob=max_ob;
80.
81. /* Free all memorys */
82.
83. free(permutation);
84. free_dictionary(pdickt);
85. free(keys);
86.
87. return OK;
88. }

```

short generate_search_times(pfunc_search method, pfunc_key_generator generator, char order, char* file, int num_min, int num_max, int incr, int n_times);

```

1. short generate_search_times(pfunc_search method, pfunc_key_generator generator, char
order, char* file, int num_min, int num_max, int incr, int n_times)
2. {
3.     PTIME_AA array_time = NULL;
4.     int i;
5.     int array_size;
6.     int N;
7.     short result;
8.
9.     assert(method != NULL);
10.    assert(generator != NULL);
11.    assert(file != NULL);
12.    assert(num_min < num_max);
13.
14.    /*Allocate memory for table of measurement*/
15.    array_size = (num_max - num_min) / incr + 1;
16.    array_time = malloc(sizeof(TIME_AA)*array_size);
17.    if (array_time == NULL)
18.    {
19.        return ERR;
20.    }
21.
22.    /*process of measurement*/
23.    for (i = 0, N = num_min; N <= num_max; i++, N+=incr)
24.    {
25.        if (average_search_time(method, generator, order,N, n_times, &(array_time[i])) ==
ERR)
26.        {
27.            free(array_time);
28.            return ERR;
29.        }
30.    }

```



```

31.
32.  /*save the result into a file*/
33.  result = save_time_table_search(file,array_time, array_size);
34.
35.  /*free memory of table*/
36.  free(array_time);
37.
38.  return result;
39. }

```

short save_time_table_search(char* file, PTIME_AA time, int N);

```

1.  short save_time_table_search(char* file, PTIME_AA time, int N)
2.  {
3.      FILE *pf = NULL;
4.      int i;
5.
6.      assert(file != NULL);
7.      assert(time != NULL);
8.      assert(N > 0);
9.
10.     /*open the file*/
11.     pf = fopen(file,"w");
12.     if (pf == NULL)
13.     {
14.         return ERR;
15.     }
16.
17.     /*print result of measurement into file*/
18.     fprintf(pf, "N Time average_ob max_ob min_ob \n");
19.
20.     for (i = 0; i < N; i++)
21.     {
22.         fprintf(pf, "%d %.10f %.10f %d %d \n", time[i].N, time[i].time, time[i].average_ob,
time[i].max_ob, time[i].min_ob);
23.     }
24.
25.     /*close file*/
26.     fclose(pf);
27.
28.     return OK;
29. }

```

5. Resultados, Gráficas

5.1 Apartado 1

Resultados del apartado 1.

1. Running exercise1
2. Praticice number 3, section 1
3. Done by: Shaofan Xu y Alejandro Zheng
4. Group: 120/127
5. Key 10 found in position 15 in 16 basic op.

```

1. Running exercise1
2. Praticice number 3, section 1
3. Done by: Shaofan Xu y Alejandro Zheng
4. Group: 120/127
5. Key 10 found in position 30 in 31 basic op.

```

```

1. Running exercise1
2. Praticice number 3, section 1
3. Done by: Shaofan Xu y Alejandro Zheng
4. Group: 120/127
5. Key 10 found in position 60 in 61 basic op.

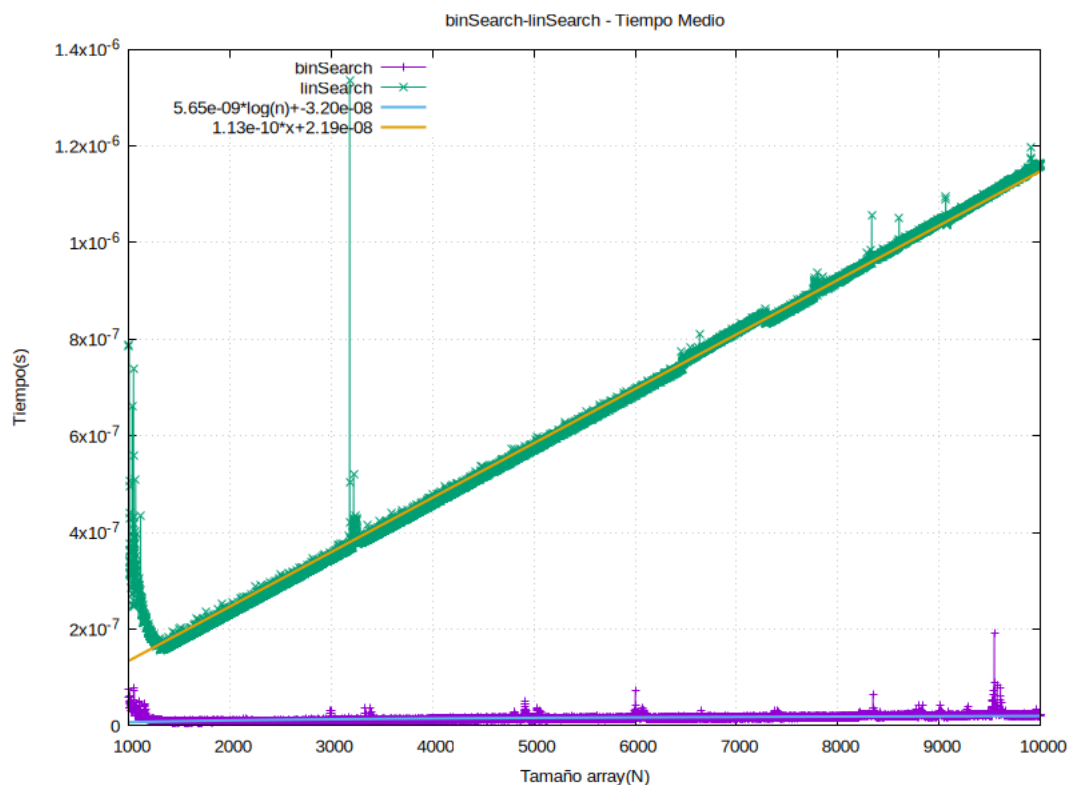
```

Así son algunos ejemplos de la salida de ejercicio1.c para verificar el correcto funcionamiento de nuestra implementación de la primera parte de práctica.

5.2 Apartado 2

Resultados del apartado 2.

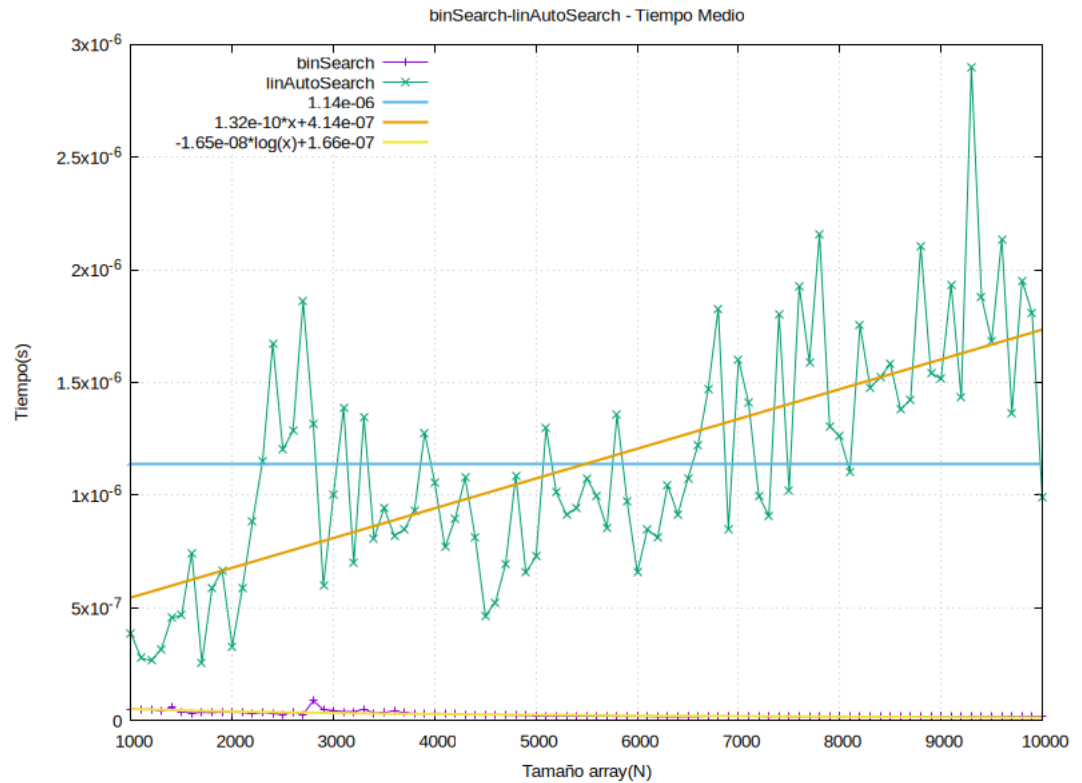
Gráfica comparando el tiempo promedio de reloj entre la búsqueda lineal y la búsqueda binaria, comentarios a la gráfica.



Para este caso, la comparación del tiempo promedio de reloj entre la búsqueda lineal y la búsqueda binaria, al parecer la búsqueda lineal tiene un rendimiento $O(N)$ y la búsqueda binaria tiene un rendimiento " $O(1)$ " teniendo un rendimiento teórico $O(\log N)$, sin embargo, ajustando con funciones adecuadas como muestra en la imagen con una recta para búsqueda lineal y una función logarítmica para la búsqueda binaria nos verifican el rendimiento teórico en ambos algoritmos. Lo cual el rendimiento medido acuerdo con el rendimiento teórico: $O(N)$ para `lin_search` y $O(\log N)$ para `bin_search`.

Gráfica comparando el tiempo medio de reloj entre la búsqueda binaria y la búsqueda lineal auto organizada (para los valores de n_times=1, 100 y 10000), comentarios a la gráfica.

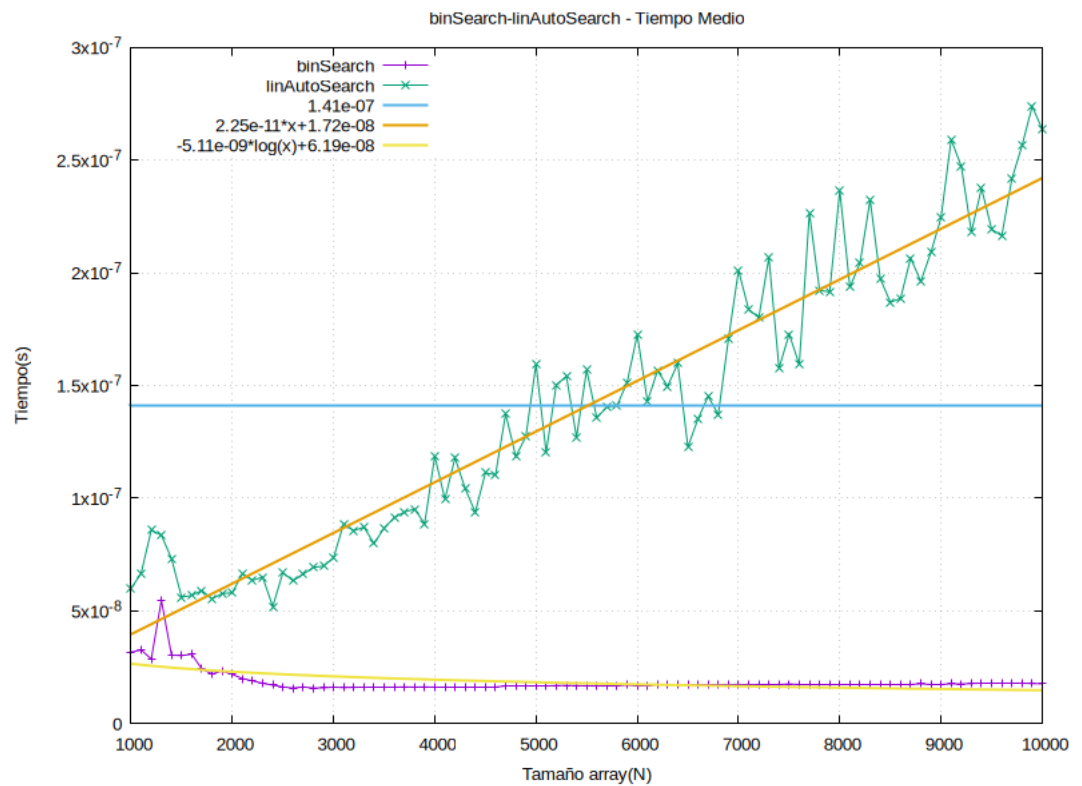
1. n_times=1



La imagen muestra la comparación entre **búsqueda binaria** y **búsqueda lineal auto-organizado** con $n_times=1$ usando **potencial_key_generador** para generar las claves a buscar, en caso de búsqueda binaria al igual que la comparación con la **búsqueda lineal** pareciendo $O(1)$ pero ajustando la gráfica con una función logarítmica nos verifica el rendimiento teórico $O(\log N)$.

Y en este caso la **búsqueda lineal auto-organizado** teniendo un rendimiento teórico casi $O(1)$ en caso de buscar las claves con mayor frecuencias, sin embargo, no es el caso para $n_times=1$, ya que dicho algoritmo auto-organiza los datos (dicho de otra manera, ordena los datos más buscados de forma que facilita la búsqueda), pero la cantidad que búsqueda que hacemos no es suficiente para que el algoritmo ordene el diccionario, lo cual, aparece un comportamiento similar a la **búsqueda lineal** $O(N)$, justificada con un ajuste lineal (podemos ver que si ajustamos con un constante, es una mala aproximación).

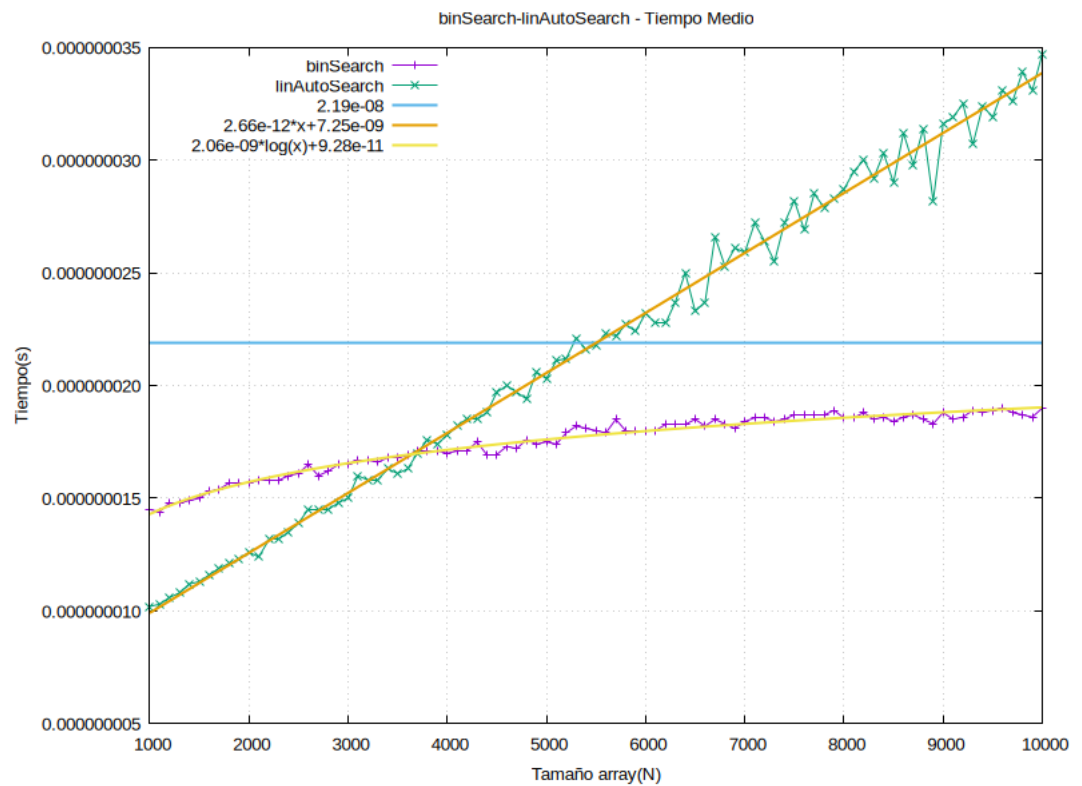
2. $n_times=100$



Nos situamos con $n_times=100$ (cada clave busca 100 veces), nos ayuda a comprender de forma mejor el comportamiento de cada algoritmo, para **búsqueda binaria**, al igual que el caso anterior su comportamiento es $O(\log N)$, justificada con un ajuste logarítmica.

Y en este caso la **búsqueda lineal auto-organizado** al buscar más veces la misma clave, facilita las búsquedas posteriores para las claves que aparecen con mayor frecuencia, lo cual podemos observar en la gráfica, que, aunque sigue pareciendo un rendimiento $O(N)$, sin embargo, el tiempo de ejecución podemos observar que se ha bajado un orden de magnitud con respecto con $n_times=1$, de un medio de 10^{-6} a 10^{-7} . Lo cual, fue bastante más rápido.

3. $n_times=10000$



Nos situamos con $n_times=10000$ (cada clave busca 10000 veces), al igual que casos anteriores, la **búsqueda binaria** tiene un rendimiento $O(\log N)$, justificada con ajusta logarítmica.

La **búsqueda lineal auto-organizado** podemos notar que al buscar la misma clave 10000 veces, su rendimiento bajo generador de clave no uniforme comporta mejor en cierto intervalo que búsqueda binaria, además, en este caso el algoritmo fue mucho mejor, bajo el tiempo de ejecución de orden de 10^{-7} a 10^{-8} .

5. Respuesta a las preguntas teóricas.

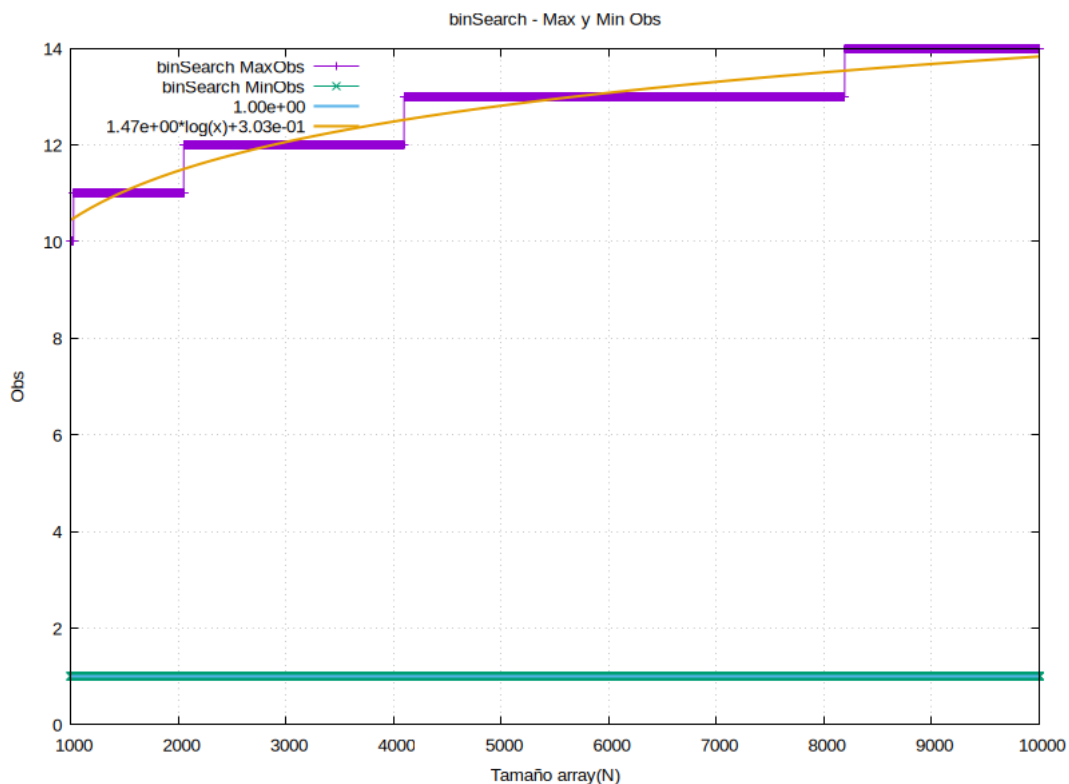
5.1 ¿Cuál es la operación básica de `bin_search`, `lin_search` y `lin_auto_search`?

Para **`bin_search`** la operación básica la comparación del índice del extremo derecho (right) y del extremo izquierdo (left) del intervalo de cada iteración que estamos trabajando.

En el caso de **`lin_search`**, la operación básica es la comparación del elemento de diccionario con la clave `if (table[i] == key)`.

Y en el caso de que **`lin_auto_search`**, al igual que **`lin_search`** la operación básica también es la comparación del elemento de diccionario con la clave.

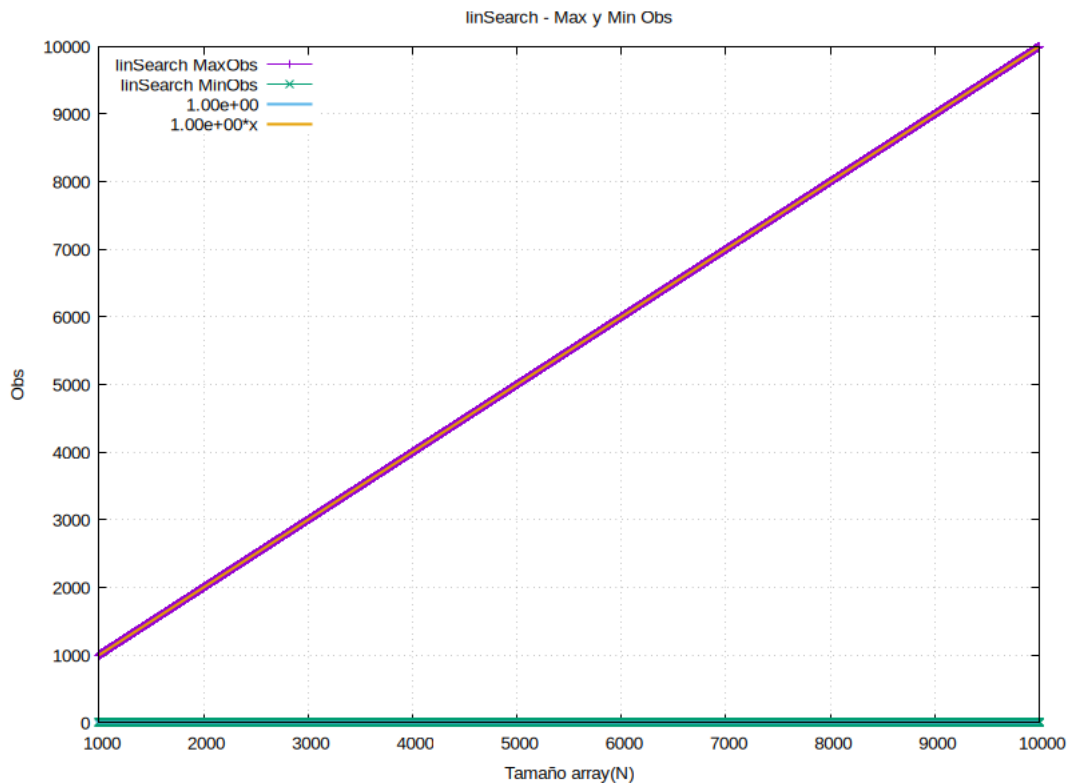
5.2 Dar tiempos de ejecución en función del tamaño de entrada n para el caso peor $WSS(n)$ y el caso mejor $BSS(n)$ de `bin_search` y `lin_search`. Utilizar la notación asintótica (O , Θ , o , Ω , etc) siempre que se pueda.



La búsqueda binaria, podemos observar que para el caso peor $WSS(n)$ presenta una gráfica con puntos discontinuos (ya que los datos son discretos, enteros), sin embargo, da una buena aproximación con ajuste logarítmica así que verifica su rendimiento teórico en caso peor, es decir, $O(\log N)$.

Y para el caso mejor, la búsqueda binaria, presenta una “línea constante” de $O(1)$, sin embargo, ajustando también con función constante también nos verifica su rendimiento para el caso mejor, lo cual, $BSS(n) = O(1)$. Y en este caso, es importante prestar una atención especial a pesar de su comportamiento casi como constante, sin embargo, es razonable, ya que al buscar

tantas veces, puede que coincidan que la clave a buscar justamente se encuentra en el centro del diccionario.



Nos situamos a analizar el funcionamiento de búsqueda lineal, a pesar de ser un algoritmo básico:

En el caso peor, el algoritmo como mucho recorre todo el diccionario para comparar con la clave, lo cual, si el diccionario es de dimensión N , como mucho tendrá N operación básica, así ajustando con una recta, obtenemos que $WSS(n)=O(N)$.

Y el caso mejor, es el caso donde la clave que queremos buscar se encuentra en al principio del diccionario, lo cual tendrá un coste de $O(1)$, luego ajustando con una función constante, nos verifica su rendimiento en caso mejor, $BSS(n)=O(1)$.

5.3 Cuando se utiliza `lin_auto_search` y la distribución no uniforme dada ¿Cómo varía la posición de los elementos de la lista de claves según se van realizando más búsquedas?

Podemos observar cada vez que se encuentra una clave se intercambia la posición con el elemento inmediatamente anterior (salvo primero elemento), lo cual, a medida que va avanzando el programa, las claves más buscadas se encontrarán al principio del diccionario.

5.4 ¿Cuál es el orden de ejecución medio de `lin_auto_search` en función del tamaño de elementos en el diccionario `n` para el caso de claves con distribución no uniforme dado? Considerar que ya se ha realizado un elevado número de búsquedas y que la lista está en situación más o menos estable.

En este caso se tiende a $O(1)$, ya que, al realizar un elevado número de búsquedas, las claves buscadas con más frecuencias se encuentran al principio del diccionario, lo cual, facilita las búsquedas posteriores.

5.5 Justifica lo más formalmente que puedas la corrección (o dicho de otra manera, el por qué busca bien) del algoritmo `bin_search`.

El algoritmo de búsqueda binaria es correcto, ya que, dado un diccionario ordenado, podemos comparar el elemento que está en la posición media del diccionario con la clave, y nos encontramos con 3 casos:

1. La clave coincide con dicho elemento, entonces devolvemos la posición y se acaba.
2. La clave es estrictamente mayor que el dicho elemento, entonces tenemos que actualizar el intervalo que estamos mirando, pues que, el índice izquierdo (el extremo izquierdo del intervalo) sea igual a $mid + 1$.
3. La clave es estrictamente menor que dicho elemento, lo cual, se actualiza el índice derecho (el extremo derecho intervalo) sea igual a $mid - 1$.

Y este proceso se ejecuta tantas veces necesarias hasta que se encuentre la clave que estamos buscando. Sin embargo, si al final de este algoritmo no encontramos la clave que queremos, entonces, dicha clave no está en el diccionario.

6. Conclusiones finales.

En esta práctica hemos implementado tres tipos de algoritmo de búsqueda y hemos medido los rendimientos de cada algoritmo. La búsqueda lineal tiene un rendimiento peor de $O(N)$ en la práctica que coincide con lo que hemos estudiado en la teoría. Asimismo, para la búsqueda binaria el resultado obtenido también es coherente con la teoría, un rendimiento de $O(\log N)$. Para la búsqueda lineal auto-organizado, al principio vemos que tiene un rendimiento de $O(N)$ cuando el número de veces que buscamos un determinado clave es 1, pero luego el tiempo que tarda a busca una clave se va reduciendo según aumentamos el número de veces que buscamos esa clave. En consecuencia, si aumentamos ese número a una cifra muy grande, el rendimiento de búsqueda lineal-auto va a tender a $O(1)$.

En conclusión, mediante la implementación en práctica y el uso de las herramientas tecnológicas, hemos logrado a comprobar los rendimientos de teórico de los tres algoritmos mediante gráficas y comparaciones de forma experimental.